

Apparel Inventory Management System

Somaan Abbasi CS-23101

Hamza Sheikh CS-23126

Ammar Ahmed CS-23127

Showzab Medhi CS-23128

Department: CIS

Course: Database Management Systems

Code: CS-222

Submitted to: Sir Muzammil & Sir Ali Akhtar

Abstract

This project presents the design and development of an Apparel Inventory and Management System tailored for a clothing retail store. The system leverages database management principles to streamline and automate daily operations, including inventory tracking, sales processing, customer management, and employee role handling. By integrating a centralized database, the system efficiently manages data related to products, suppliers, customers, and transactions.

Key functionalities include real-time stock monitoring, automated alerts for low inventory, generation of sales reports, and tracking of customer purchase history. The system also incorporates role-based access control to ensure secure operations and limit functionalities based on user roles. A user-friendly interface allows staff to perform essential tasks such as searching products by category, size, or price, and managing supplier and employee information.

The project emphasizes both usability and performance., ultimately aiming to enhance operational efficiency, reduce manual errors, and support better decision-making through organized data management and insightful reporting.

Table of Contents

Chapter 1: Project Overview	4
Chapter 2: Requirement Formulation and Analysis	6
Chapter 3: Conceptual Design.....	8
Chapter 4: Logical Design	10
Chapter 5: Implementation Design	12
Chapter 6: Implementation of Front-End Application.....	14
Chapter 7: Database Security Implementation and Future Improvements.....	18
Conclusion	22

Chapter 1: Project Overview

Apparel Inventory and Management System

The Apparel Inventory and Management System addresses critical challenges faced by clothing retailers in managing inventory efficiently. Using Python with Flask for the backend, HTML/CSS with Bootstrap for the frontend, and SQLAlchemy for database operations, this system streamlines inventory tracking, sales processing, customer management, and supplier interactions.

Problems to be solved

1. **Inventory Overstock:** Without real-time tracking, stores often purchase excess merchandise, tying up capital and storage space. **Stockouts:** Inadequate inventory management leads to popular items being unavailable, resulting in lost sales and customer dissatisfaction.
2. **Incorrect Inventory Records:** Manual processes create discrepancies between actual stock and recorded values.
3. **Time-Consuming Processes:** Inefficient inventory procedures reduce employee productivity and increase operational costs.
4. **Poor Reporting and Analytics:** Lack of systematic approaches hinders trend tracking and strategic decision-making.

Objectives of the Database System

1. Implement Real-Time Inventory Tracking: Provide accurate, timely updates on stock levels.
2. Optimize Stock Levels: Analyze sales data to maintain optimal inventory and generate reorder recommendations.
3. Enhance Accuracy: Reduce human errors through automation and digital record-keeping.
4. Increase Efficiency: Streamline processes to free employee time for customer service.
5. Improve Reporting and Analytics: Provide insights into inventory performance and sales trends.

Scope of the Project

The system encompasses:

- User Authentication and Authorization
- Customer Management
- Product Management
- Order Processing
- Inventory Control
- Supplier Management
- Employee Management
- Reporting and Analytics

The system is designed to be scalable, allowing for future enhancements such as integration with point-of-sale systems, e-commerce platforms, and advanced analytics tools.

Chapter 2: Requirement Formulation and Analysis

This chapter outlines the systematic approach used to formulate and analyze requirements for the Apparel Inventory and Management System, involving stakeholder identification, data requirements, and functional/non-functional requirements.

Requirement Formulation

1. Stakeholder Identification

Primary Stakeholders: Store Managers, Inventory Managers, Sales Associates, IT Department

Secondary Stakeholders: Customers, Suppliers, Accountants

2. Data Requirements

Key data elements include:

- Product Information: ID, name, category, price, quantity, description, supplier information
- Customer Information: ID, name, contact details, purchase history
- Order Information: ID, customer, employee, date, payment method, amount, status, products
- Employee Information: ID, name, contact details, role, salary, hire date
- Supplier Information: ID, name, contact details, products supplied

3. Functional Requirements

- Inventory Management: Track stock levels, set reorder points, generate alerts, record movements
- Customer Management: Maintain customer records, track purchase history
- Order Processing: Create orders, calculate totals, process payments, update inventory
- Employee Management: Maintain records, assign roles, track performance
- Supplier Management: Maintain records, track orders, evaluate performance
- Reporting and Analytics: Generate inventory and sales reports, analyze performance

4. Non-Functional Requirements

- Performance: Handle 1000 transactions/day, quick query response
- Scalability: Accommodate growth in products, customers, and transactions
- Reliability: High uptime, regular backups, recovery mechanisms
- Security: Authentication, access control, data protection
- Usability: Intuitive interface, responsive design, helpful guidance

Requirement Analysis

1. Use Case Analysis

Key use cases include adding products, processing sales, and generating reports, each with defined actors, preconditions, flows, and postconditions.

2. Entity-Relationship Diagram (ERD)

The ERD identifies entities (Customer, Product, Order, Employee, Supplier, User) and their relationships, forming the foundation for database design.

3. Data Flow Diagram (DFD)

The DFD illustrates how data moves through the system, showing external entities, processes, data stores, and data flows.

4. Feasibility Analysis

The project is technically, economically, operationally, and schedule feasible, with appropriate technologies, reasonable costs, alignment with business operations, and achievable timeframes.

Chapter 3: Conceptual Design

The conceptual design phase translates requirements into a high-level representation of the database structure, serving as a blueprint for logical and physical database design.

Entity-Relationship Diagram

Entities and Attributes

1. Customer: customer_id (PK), name, email, phone_no, address, created_at
2. Product: product_id (PK), name, price, quantity_in_stock, category, description, image_url, created_at
3. Order: order_id (PK), date, payment_method, amount, status, customer_id (FK), employee_id (FK)
4. Employee: employee_id (PK), name, email, phone_no, role, salary, hire_date
5. Supplier: supplier_id (PK), name, phone_no, email, address
6. User: user_id (PK), username, password_hash, is_admin, employee_id (FK)

Relationships

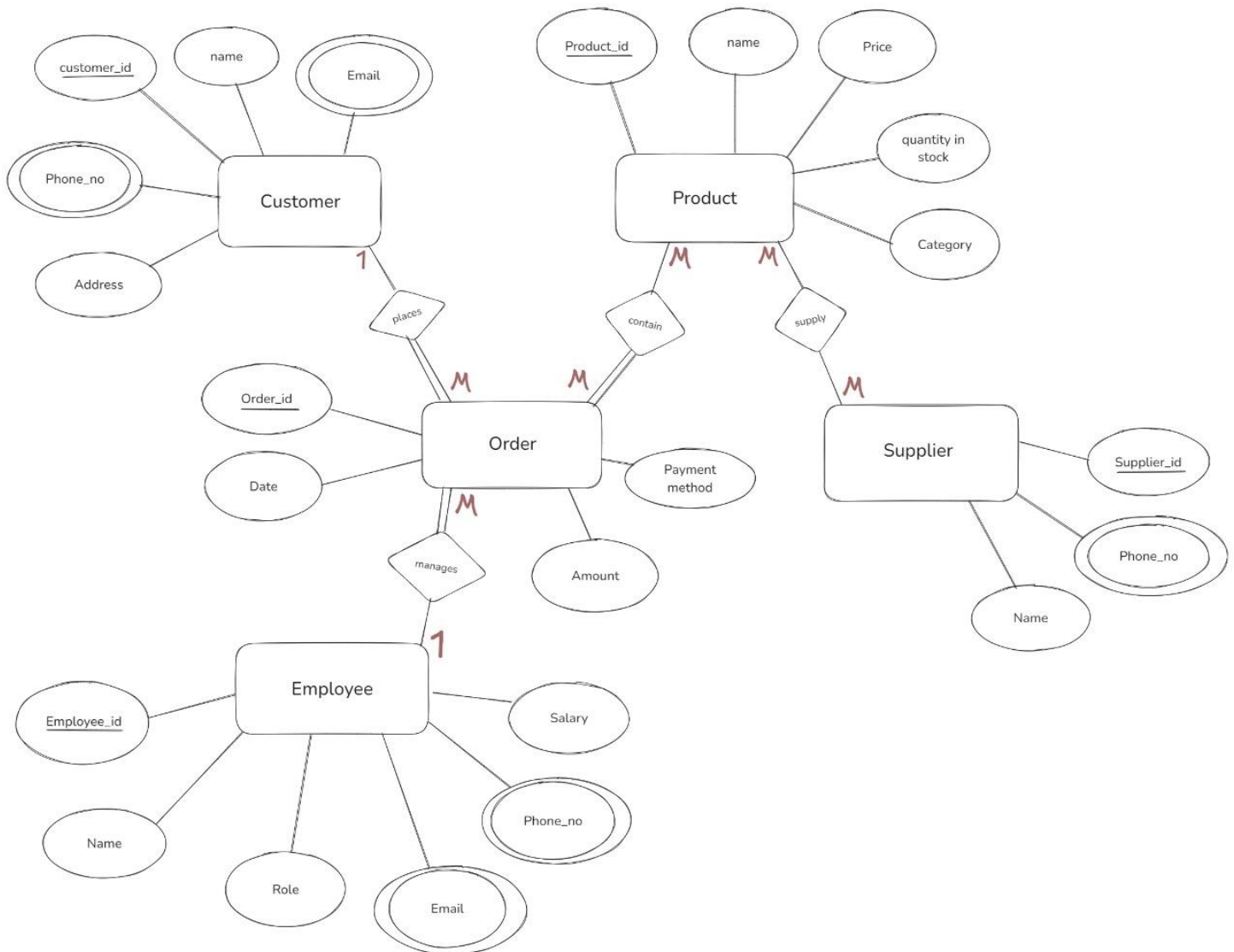
1. Customer and Order: One-to-Many (1:N) - A customer can place multiple orders
2. Employee and Order: One-to-Many (1:N) - An employee can manage multiple orders
3. Order and Product: Many-to-Many (M:N) - An order can contain multiple products
4. Product and Supplier: Many-to-Many (M:N) - A product can be supplied by multiple suppliers
5. Employee and User: One-to-One (1:1) - An employee can have one user account

Business Rules

Key business rules include:

- Customers and employees must exist before being associated with orders
- Products must be in stock to be added to orders
- Order processing automatically reduces product quantity
- Each user account must have a unique username
- Different access levels for admin and regular users
- Each product must have category, name, and price
- Order status can be "Pending," "Completed," or "Cancelled"
- Each order must contain at least one product

The conceptual design has been validated to ensure all required entities and attributes are represented, relationships accurately reflect business processes, and the schema supports all identified use cases and future enhancements.



Chapter 4: Logical Design

The logical design phase transforms the conceptual model into a detailed database structure for implementation in a relational database management system, focusing on normalization, table definitions, and constraints.

Normalization Process

The database is normalized to the Third Normal Form (3NF):

1. First Normal Form (1NF): Ensures atomic values, primary keys, and no repeating groups

2. Second Normal Form (2NF): Ensures non-key attributes depend on the entire primary key
3. Third Normal Form (3NF): Eliminates transitive dependencies between non-key attributes

Relational Schema

Tables and Attributes

1. Customer: id (PK), name, email, phone_no, address, created_at
2. Product: id (PK), name, price, quantity_in_stock, category, description, image_url, created_at
3. Order: id (PK), date, payment_method, amount, status, customer_id (FK), employee_id (FK)
4. OrderProduct: order_id (PK, FK), product_id (PK, FK), quantity, price_at_purchase
5. Employee: id (PK), name, email, phone_no, role, salary, hire_date
6. Supplier: id (PK), name, phone_no, email, address
7. ProductSupplier: product_id (PK, FK), supplier_id (PK, FK)
8. User: id (PK), username, password_hash, is_admin, employee_id (FK)

Integrity Constraints

1. Entity Integrity: Primary keys defined for all tables and cannot be null
2. Referential Integrity: Foreign keys enforce relationships between tables
3. Domain Integrity: Data types, NOT NULL constraints, DEFAULT values, and CHECK constraints

Indexes

Indexes are defined to improve query performance:

- Primary key indexes (automatic)
- Foreign key indexes
- Additional indexes on frequently queried columns (product category, product name, order date, etc.)

Denormalization Considerations

Strategic denormalization is applied for performance reasons:

- Storing order total amount in the Order table
- Keeping current price in Product table while also storing price_at_purchase in OrderProduct

These optimizations improve query performance while maintaining data integrity through application logic.

Chapter 5: Implementation Design

The implementation design phase translates the logical database design into a physical implementation using SQLAlchemy ORM with SQLite as the database engine.

Database Management System Selection

SQLite was chosen for development due to its:

- Simplicity and zero configuration
- Self-contained, serverless nature
- Reliability and cross-platform compatibility
- Suitability for development and testing

For production, the system can be migrated to PostgreSQL or MySQL by changing the connection string.

Physical Schema Implementation

The database schema is implemented using SQLAlchemy's Object-Relational Mapping (ORM), which provides a high-level, object-oriented interface to the database. Models are created for Customer, Product, Order, Employee, Supplier, and User entities, with appropriate relationships defined between them.

Data Access Methods

The system implements Create, Read, Update, and Delete (CRUD) operations for all entities:

- Create: Adding new records to the database
- Read: Retrieving and displaying records
- Update: Modifying existing records
- Delete: Removing records from the database

Query Optimization

Several techniques are employed to optimize database queries:

- Eager Loading: Reducing the number of database queries when retrieving related objects
- Indexing: Creating indexes on frequently queried columns
- Query Caching: Caching results of frequently accessed, relatively static data

Transaction Management

Transactions ensure data integrity for operations affecting multiple records:

- Beginning transactions before complex operations
- Committing changes when operations complete successfully
- Rolling back changes when errors occur

Database Security

Security measures at the database level include:

- Parameterized Queries: Preventing SQL injection
- Password Hashing: Securing user passwords
- Role-Based Access: Restricting database operations based on user roles
- Input Validation: Validating all user inputs
- Error Handling: Catching and handling exceptions appropriately

Chapter 6: Implementation of Front-End Application

The front-end application is implemented using Flask as the web framework, with HTML, CSS, and JavaScript for the user interface, focusing on usability, responsiveness, and functionality.

Front-End Architecture

Template Structure

The application uses Flask's Jinja2 templating engine with a hierarchical structure:

- Base Template: Contains common layout elements (navigation, header, etc.)
- Content Templates: Extend the base template for specific functionality
- Form Templates: Dedicated templates for data entry

Responsive Design

The front-end uses Bootstrap 5 for responsive design:

- Grid System: 12-column grid for layout
- Responsive Components: Tables, forms, and cards adapt to different screen sizes
- Mobile Navigation: Collapsible sidebar for mobile devices
- Media Queries: Custom CSS for specific screen size adjustments

User Interface Components

Dashboard

The dashboard provides an overview of key metrics:

- Metric Cards: Counts for customers, products, orders, and low stock items
- Recent Orders Table: Most recent orders with key details
- Quick Action Buttons: Shortcuts to common tasks

Apparel Inventory

Dashboard

Customers

Products

Orders

Suppliers

View Shop

Employees

admin

Dashboard

Login successful!

CUSTOMERS4

View Details

PRODUCTS3

View Details

ORDERS4

View Details

LOW STOCK2

View Details

Recent Orders

Order ID	Customer	Date	Amount	Status
4	Hamza	2025-05-17 18:22	\$2.00	Pending
3	Ammar Ahmed	2025-05-15 17:40	\$23.00	Pending
2	Ammar Ahmed	2025-05-15 17:39	\$23.00	Cancelled
1	Soman	2025-05-14 17:28	\$6.00	Completed

Quick Actions

Add Customer

Add Product

Create Order

Add Supplier

View Online Store

Data Tables

Data tables display listings of customers, products, orders, etc.:

- Responsive Tables: Scroll horizontally on small screens
- Sorting and Filtering: Sort columns and filter content
- Action Buttons: Edit, delete, and view buttons for each row

Apparel Inventory

Dashboard

Customers

Products

Orders

Suppliers

View Shop

Employees

admin

Customers

Add Customer

ID	Name	Email	Phone	Address	Actions
2	Soman	soman@gmail.com	175	A-138 Block 14 Gulistan-e-Ikhar Karachi	

3	Ammar Ahmed	ammara1234@gmail.com	0332018760	A-138 Block 14 Gulistan-e-Ikhar Karachi	
4	Hamza	hamza@gmail.com			
5	Hamza	hamza@gmail.com			

Products

Add Product

ID	Name	Category	Price	Stock	Actions
1	Jeans1	Jeans	\$2.00		
2	Shirts	Shirts	\$23.00		
3	Chen	Accessories	\$2.00		

Orders

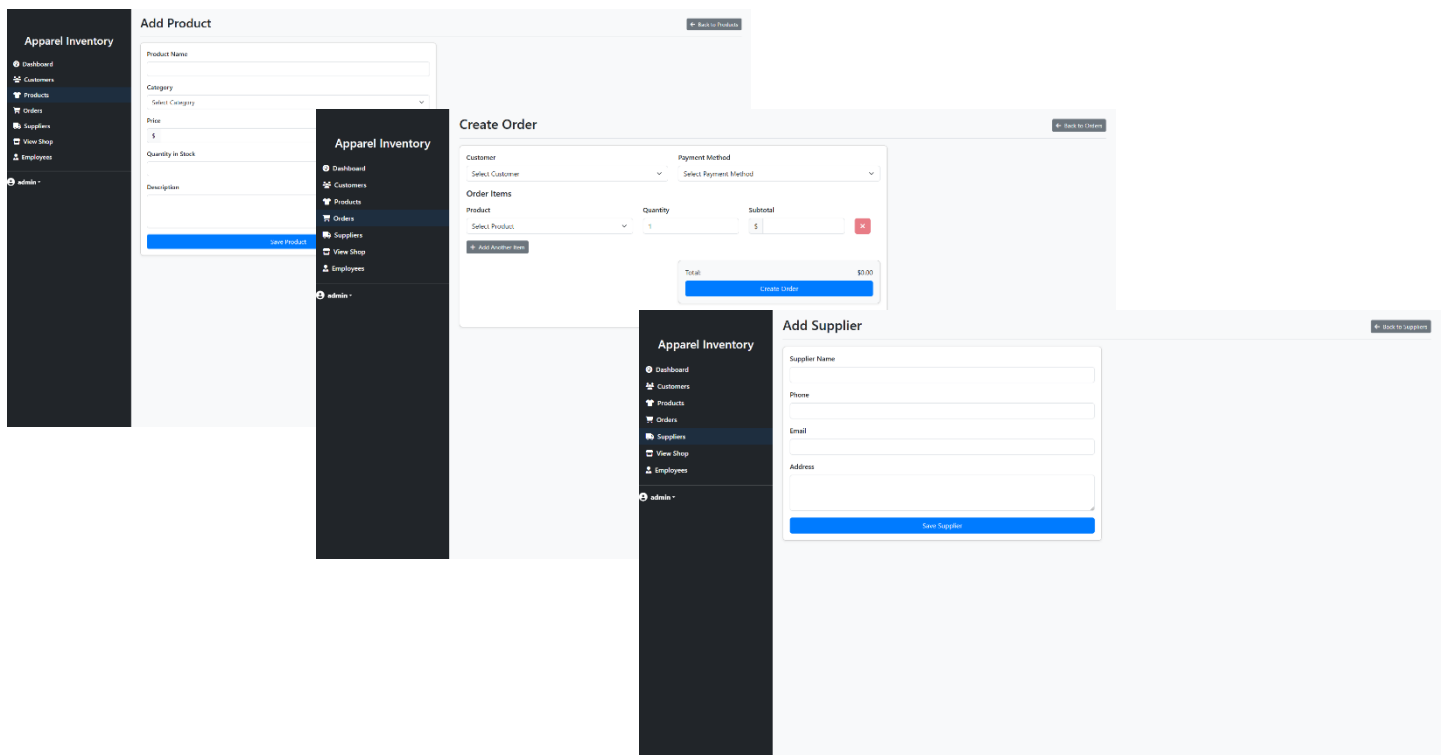
Create Order

Order ID	Customer	Employee	Date	Amount	Payment Method	Status	Actions
1	Soman	Showab	2025-05-14 17:28	\$6.00	Credit Card	Completed	
4	Ammar Ahmed	Showab	2025-05-15 17:39	\$23.00	Credit Card	Cancelled	
3	Ammar Ahmed	Showab	2025-05-15 17:40	\$23.00	Credit Card	Pending	
4	Hamza	Showab	2025-05-17 18:22	\$2.00	Credit Card	Pending	

Forms

Forms handle data entry and editing:

- Validation: Client-side and server-side validation
- Responsive Layout: Forms adapt to different screen sizes
- Input Types: Appropriate input types for different data
- Error Handling: Display validation errors



Client-Side Functionality

JavaScript enhances the user experience with:

- Form Validation: Validates user input before submission
- Dynamic Content: Updates content without page reloads
- Interactive Elements: Tooltips, popovers, and modals
- AJAX Requests: Fetches data asynchronously

Splash Screen Animation

A splash screen animation enhances the user experience:

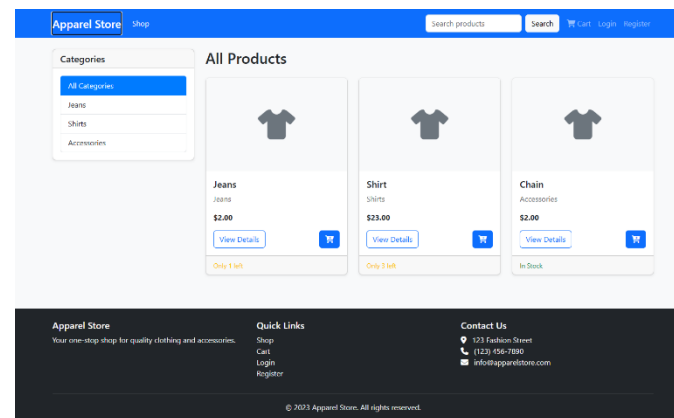
- Video Background: Short, looping video related to apparel retail
- Fade-In Effect: Content smoothly fades in after the splash screen
- Session-Based Display: Only shows on the first visit in a browser session

Form Handling and Validation

Server-Side Validation

Form data is validated on the server to ensure data integrity:

- Required Fields: Checks that all required fields are provided
- Data Types: Validates that data is of the correct type
- Business Rules: Enforces business rules
- Error Messages: Returns clear error messages when validation fails

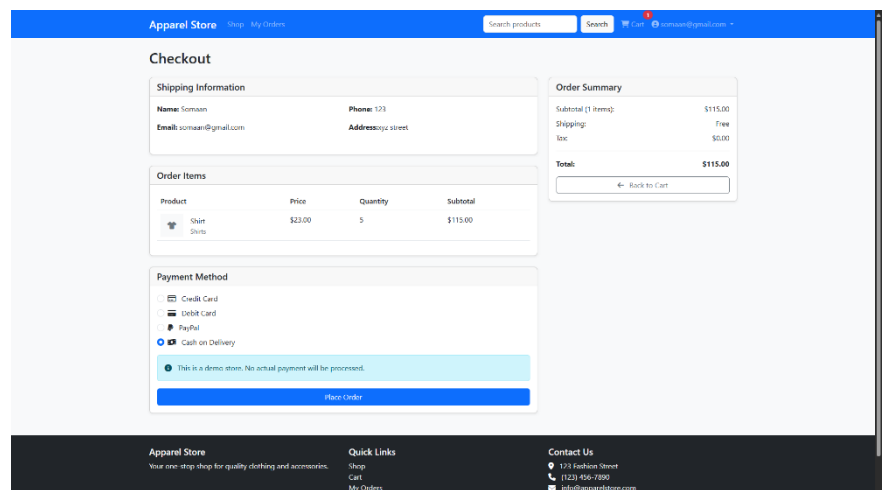


Client-Side Validation

Client-side validation improves user experience:

- HTML5 Validation: Uses built-in validation attributes
- JavaScript Validation: Adds custom validation logic
- Visual Feedback: Highlights invalid

fields and displays error messages



Chapter 7: Database Security Implementation and Future Improvements

This chapter outlines the security measures implemented in the system and discusses potential future improvements to enhance security, functionality, and performance.

Current Security Measures

1. User Authentication

- Password Hashing: Securely hashes passwords with unique salts
- No Plaintext Storage: Passwords are never stored in plaintext
- Secure Sessions: Uses Flask-Login for secure session management

2. Role-Based Access Control

- Admin Role: Administrators have access to all system functions
- Employee Role: Regular employees have limited access
- Route Protection: Sensitive routes are protected with role checks
- UI Adaptation: User interface shows only accessible functions

3. Input Validation and Sanitization

- Form Validation: Validates all form inputs
- Parameterized Queries: Uses SQLAlchemy's ORM to prevent SQL injection
- Data Type Checking: Ensures data is of the expected type
- Length Limits: Enforces maximum lengths for string inputs

4. CSRF Protection

- CSRF Tokens: Generates and validates tokens for all forms

- Per-Session Tokens: Each user session has unique tokens
- Token Validation: Validates tokens on all state-changing requests

5. Error Handling and Logging

- Custom Error Pages: Provides user-friendly error pages
- Exception Handling: Catches and handles exceptions appropriately
- Error Logging: Logs errors for later analysis
- User Feedback: Provides appropriate feedback without exposing sensitive information

Future Security Improvements

1. Enhanced Authentication

- Multi-Factor Authentication: Implement 2FA using email, SMS, or authenticator apps
- Advanced Password Policies: Enforce stronger password requirements
- Account Lockout: Implement lockout after failed login attempts

2. Improved Data Protection

- Field-Level Encryption: Encrypt sensitive fields like credit card numbers
- Data Masking: Mask sensitive data in production environments
- Data Minimization: Collect and store only necessary data

3. Advanced Monitoring and Auditing

- Detailed Audit Logs: Log all significant actions
- Real-Time Monitoring: Implement intrusion and anomaly detection
- Alert System: Set up alerts for suspicious activities

Functional Improvements

1. Advanced Reporting and Analytics

- Sales Analytics: Implement advanced sales analytics with visualizations
- Inventory Forecasting: Add predictive analytics for inventory needs
- Customer Insights: Analyze customer purchasing patterns

2. Barcode/QR Code Integration

- Barcode Scanning: Add support for barcode scanning
- QR Code Generation: Generate QR codes for products and orders
- Mobile Integration: Develop mobile app for scanning

3. Customer Relationship Management

- Loyalty Program: Implement customer loyalty points system
- Customer Segmentation: Segment customers based on behavior
- Personalized Marketing: Enable targeted promotions

4. Integration Capabilities

- E-commerce Integration: Connect with online store platforms
- Payment Gateway Integration: Integrate with payment processors
- Accounting Software Integration: Connect with accounting systems

Performance Improvements

1. Database Optimization

- Query Optimization: Optimize complex queries
- Index Tuning: Fine-tune database indexes
- Connection Pooling: Implement connection pooling

2. Caching Strategies

- Result Caching: Cache query results for frequently accessed data
- Fragment Caching: Cache portions of rendered templates
- Distributed Caching: Implement Redis or Memcached

3. Asynchronous Processing

- Background Tasks: Use Celery for time-consuming operations
- Scheduled Jobs: Implement scheduled tasks for regular operations

4. Frontend Optimization

- Lazy Loading: Implement lazy loading for images and content
- Code Splitting: Split JavaScript code for faster initial loading
- Asset Optimization: Optimize images and other assets

Conclusion

The Apparel Inventory and Management System provides a comprehensive solution for clothing retail stores. Current implementation includes robust security measures, while future improvements will focus on enhancing security, functionality, and performance to meet evolving business needs.